

Dot1Q Code Flow

The VLAN protocol folder consists within it, modules like the Dot1q, Dot1v, ipsubnetvlan and Macvlan.

DOT1Q (IEEE 802.1Q)

Which is the Standard VLAN tagging method. Here, when a frame travels across a switch trunk, the switch inserts a **4-byte tag** containing information regarding

- **TPID (Tag Protocol Identifier):** 0x8100
- **TCI (Tag Control Information):**
 - **PCP:** priority (QoS). **(3 bits)**
 - **DEI:** drop eligibility **(1 bit)**
 - **VID:** VLAN ID **(12 bits)**

DOT1Q enables multiple VLANs per trunk and separation of broadcast domains.

Code Flow:

Step 1: Entering the CLI command → the CLI tree:

We can use the example of a CLI command `vlan ingressfilter` to configure ingress filtering.

Any CLI command to configure VLAN lands on the

`mgmt/broadcom/switching/vlan/` folder which contains the APIs that will in turn call the specific protocol. This happens via the CLI parse tree built of the VLAN cli commands

After checking if the feature is present through the usmDb layer via

`usmDbFeaturePresentCheck` for that particular chipset (unit) the buildTree function for that particular module calls the Cli function as mentioned below for this example.

`cliCommandsInit` (inside `/mgmt/broadcom/cli/base/common/clicommands.c`) . This checks if the command/ module specific macro is Set (here, `L7_ROUTING_PACKAGE`) and calls the buildTree function for this command

(`buildTreeInterfaceVlanConfig`) →

`buildTreeInterfaceCommonConfig` →

`buildTreeInterfaceSwDevVlan`

`buildTreeInterfaceSwDevVlan` → under depth 3 of this CLI tree calls the
vlan ingress filter cli function `commandVlanIngressFilter` through
`ewsCliAddNode` if feature is allowed for chipset under `usmDb`

`buildTreeInterfaceVlanConfig` →

`buildTreeInterfaceCommonConfig` →

`buildTreeInterfaceSwDevVlan` → `commandVlanIngressFilter`

Step 2: CLI API calls the actual module or protocol_api:

In this example, `commandVlanIngressFilter` is a part of the protocol specific CLI
landing function within `src/mgmt/broadcom/cli/switching/vlan/dot1q/cli_config_vlan.c`

Every CLI command specific to VLAN 802.1q lands on the dot1q folder based on whether it is
show command for .1q or vlan or a config operation for .1q or for vlan.

These api also checks the cli arguments and values, the prompt, the mode and returns error
if incorrect. It receives the structure of current cli context (`ewsContext`). It also writes the
output back into the cli by returning functions like

`cliSyntaxReturnPromptAddBlanks` (if error) / `cliPrompt` if executed.

Within this, it calls the **`usmDb`** to set the ingressFiltering in this case. For other cli commands,
it can call functions directly from NIM if ports are involved to check the interface type and if it
is within specified range before proceeding to execute the command.

The `usmDb` lies between the CLI layer and the actual protocol. Its functions generally tend to
check if the configuration can be done on some module, if the support is there. For example,
it can get the interface type from NIM and check if it is valid before proceeding with ingress
tagging. All the functions here call the `protocol_api` for which the config is intended.

`usmDbQportsEnableIngressFilteringSet` →

`dot1qQportsIngressFilterSet` (part of `dot1q_api.c`)

Step 3: Inside the module : /application/switching/vlan/dot1q/dot1q_api.c

The dot1q folder (or any module folder) has several files each separate for:

- Data addition / deletion - `data.c` (calls to trees for tables)
- `control.c` for process control like message queues send and recv , getting events, to notify registered users, etc.

- debug.c to print the events like number of messages in queue, to reset counters in queues, debug trace mode, incase of events - print event specific messages like `dot1q_vlan_interface_delete` or `Unknwon event` etc.
- qports.c - all calls related to interfaces
- syncdb.c - to store AVL tree backup and retrieve as necessary. functions to print trace messages incase of failures. functions here map to the syncdb_api inside the process/proc_libs (here sockets can be created to send messages to the syncdb process or recv messages from the sockets). calls like `syncdbAvlTableCreate` to the syncdb.api which inturn creates a backup syncdb AVL tree for table info provided.
- `dot1q_api.c` : API calls to the module
- outcalls - functions for notifying all the config changes

In this example: `dot1qQportsIngressFilterSet`

The dot1q api first takes `osapiWriteLockTake` (pthread mutex lock from `osapi_rwlock.c`) , checks if its configurable (makes calls to NIM as this is port specific), updates the port config structure `dot1qIntfCfgData_s` having `L7_uchar8-enableIngressFiltering` checks if ingressTag changed and posts the message queue to the receiver via `dot1qIssueCmd` (event set as `dot1q_qports_ingress_filter_set`)

The dot1qIssueCmd sends the message using `osapiMessageSend` through the `dot1qMsgQueue` and also logs the message queue event via the Trace - `dot1qTraceQueueMsgSend`

Receiver side:

`dot1qTask` receives the queue and initiates the event as included in the message. (`osapiMessageReceive`).

This is done via the dispatcher - `dot1qDispatch`

Each event within this maps to it's specific purpose. (here, needs to perform ingress tagging)

In this case, `event = dot1q_qports_ingress_filter_set` is mapped to case `dot1qQportsIngressFilterProcess`

As this is a port specific process, the function is called from within dot1q.qports.c.

Here, interface specific checks like - `dot1qMapIntfIsConfigurable`
`dot1qIntfIsSettable` `nimGetIntfName` which calls NIM before calling the DTL api to set ingress filtering on the port.

Finally calls the `dtlDot1qPortConfig` or the Device Translation Layer between the Protocol and DAPI.

Step 4: DTL → DAPI → HAPI → USL → BCM.

- Once the call reached DTL in this format `dtlDot1qPortConfig` with the parameter mask set to `DTL_L2_DOT1Q_PORT_CONFIG_INGRESS_FILTER`. It sets the `dapiCmd` struct with `ingressFilteringEnabled` as True.
- It then calls the DAPI `dapiCtl` with macro `DAPI_CMD_QVLAN_PORT_CONFIG`
- The `dapiCtl` is single point for all the DAPI calls coming from all the modules to reach the HAPI layer. (function pointers)
- `dapiPortPtr->cmdTable[DAPI_CMD_QVLAN_PORT_CONFIG] = (HAPICTLFUNCPTR_t)hapiBroadQvlanPortConfig;`
- The `cmdTable` will point to the specific HAPI func ptr, which here is `hapiBroadQvlanPortConfig` - within `M4350/andl/hapi/esw/switching/broad_l2_vlan.c`
- within the critical section of the `hapiBroadQvlanPortConfig` it first checks the `ingressFilteringEnabled` set earlier and the `ingressFilteringEnabledModify` if set to True (L7_TRUE)
`ingressFilteringEnabled = cmdVlanPortCfg->cmdData.portConfig.ingressFilteringEnabled;`
- If true: `hapiBroadVlanIngressFilterSet` is called on the given `hapiLagMemberPortPtr->hapi_broad_lport` (the interface taken from the USP info from DAPI/DTL). Within this, it checks if the `lport` input is valid `lport` and if the `dapiUsp` is valid using the `HapiPortPtr`. `BCM_PORT_VLAN_MEMBER_INGRESS` is set on the `usl_bcm_port_filter_mode_t` mode dataType with mode.value as

input ingress filter value —→ `usl_bcmx_port_vlan_member_set` on Lport for mode

- Unit Synchronization Layer: The `usl_bcmx_port_vlan_member_set` is located within the `hapi/esw/usl/port/l7_usl_bcmx_port.c` taking the `usl_bcm_port_filter_mode_t` as input for the broad-lport. It then calls the RPC (Remote Procedure Call) client api to set the Ingress filtering for the port(s) and also calls `usl_portdb_filter_mode_set` to update the filtering mode in the USL database.(`usl_portdb_port_record_find`)
- `l7_rpc_client_port_vlan_member_set` maps to the `hapiBroadCustomPortSet` which calls the BCM finally `bcm_custom_port_set` for the particular bcm unit , port.

To summarize, all the calls made to the DTL goes to DAPI (DapiCtl) which maps it to the HAPI function pointers. From HAPI, its taken to USL (sometimes Usl db is updated) and from there through RPC reaches the broadcom custom apis which makes the final bcm call.